图像膨胀测试

图像膨胀测试
- 例程效果
- 原理讲解
  - 什么是图像膨胀
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置膨胀操作参数
    - 图像处理与膨胀运算
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/19.rgb888_dilate.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现 RGB888 格式的图像膨胀处理。



原理讲解

什么是图像膨胀

图像膨胀（Dilation）是一种形态学图像处理操作，用于扩大图像中的白色区域（前景），从而填充小孔洞、连接断开的区域或增强对象的边界。膨胀操作是形态学处理的基础操作之一，常用于二值图像或灰度图像。

- **操作原理**：膨胀使用一个结构元素（通常称为卷积核或内核），在图像上滑动。如果内核覆盖区域内至少有一个像素是白色（或高值），则将内核中心位置的像素设置为白色（或最大值）。这导致白色区域向外扩展，边界变粗，小孔洞被填充。
- **效果**：膨胀操作可以使图像中的白色对象变大，连接相邻但断开的区域，并去除小的黑色噪声（如小黑点）。在实际应用中，膨胀常用于增强连通性、填充空隙或作为其他复杂形态学操作（如闭运算）的基础。
- **应用场景**：膨胀操作在图像预处理中非常常见，例如在目标检测中用于连接断开的边缘，在医学图像处理中用于填充小空洞，或在字符识别中用于增强文字的连续性。

在膨胀操作中，卷积核的大小和迭代次数会显著影响结果：较大的核和更多的迭代次数会导致更显著的扩展效果。此外，二值化阈值（如果填写了）会决定哪些像素被视为前景（白色）或背景（黑色）。

代码概述

导入模块

```
import time, os, gc
from machine import Pin
from media.sensor import *    # 摄像头接口 / Camera interface
from media.display import *   # 显示接口 / Display interface
from media.media import *     # 媒体资源管理器 / Media manager
import cv_lite                # cv_lite 扩展模块（含膨胀接口）/ AI CV extension
(dilate function)
import ulab.numpy as np       # NumPy-like ndarray for MicroPython
```

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(id=2, width=1280, height=960, fps=30)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.RGB888)  # 设置像素格式为 RGB888 / Set pixel format to
RGB888
```

- 初始化摄像头，设置分辨率为 1280x960，帧率为 30 FPS。
- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合处理彩色图像的形态学操作）。

初始化显示模块

```
Display.init(Display.ST7701, width=image_shape[1], height=image_shape[0],
to_ide=True)
```

初始化显示模块，使用 ST7701 驱动芯片，分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置膨胀操作参数

```
kernel_size = 3          # 卷积核尺寸 / Kernel size (推荐为奇数，但部分实现支持偶数)
iterations = 1           # 膨胀迭代次数 / Number of dilation passes
threshold_value = 100    # 二值化阈值（0=使用 Otsu 自动阈值）/ Threshold for
binarization (0 = Otsu)
clock = time.clock()     # 启动帧率计时器 / Start FPS timer
```

- `kernel_size`：卷积核尺寸，用于膨胀操作，建议为奇数（部分实现支持偶数），值越大扩展范围越大。
- `iterations`：膨胀操作的迭代次数，值越大扩展效果越显著，但计算量增加。
- `threshold_value`：二值化阈值，用于在膨胀前或后处理图像，值为 0 时使用 Otsu 自动阈值算法。
- `clock`：用于计算帧率。

图像处理与膨胀运算

```
while True:
    clock.tick()  # 开始计时 / Start frame timing

    # 获取一帧图像 / Capture a frame
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()  # 获取 RGB888 ndarray 引用 / Get RGB888 ndarray
reference

    # 应用膨胀操作（内部先转换为灰度再膨胀）/ Apply dilation (converts RGB to gray
internally)
    result_np = cv_lite.rgb888_dilate(
        image_shape,
        img_np,
        kernel_size,
        iterations,
        threshold_value
    )

    # 构造图像用于显示（灰度格式） / Wrap dilated grayscale image for display
    img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                          alloc=image.ALLOC_REF, data=result_np)
```

```
    # 显示膨胀处理结果 / Show dilated image
    Display.show_image(img_out)

    # 清理内存并输出帧率 / Cleanup and print FPS
    gc.collect()
    print("dilate fps:", clock.fps())
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **膨胀运算处理**：调用 `cv_lite.rgb888_dilate()` 函数执行膨胀操作（内部先将RGB图像转换为灰度图像），返回处理后的图像 NumPy 数组。
- **图像包装和显示**：将处理结果包装为灰度图像对象，并显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

参数调整说明

- `kernel_size`：卷积核尺寸，值越大扩展的区域范围越大，适合处理较大的断开区域或填充更大的孔洞；建议使用奇数（如 3、5、7），从 3 开始测试以平衡细节和扩展效果。
- `iterations`：迭代次数，值越大膨胀效果越强，但可能导致过度扩展和细节丢失；建议从 1 开始，根据图像特性逐步增加。
- `threshold_value`：二值化阈值，值越高需要更亮的像素才能被视为前景，值为 0 时使用 Otsu 自动阈值；建议根据图像亮度从 50 到 150 测试，或设置为 0 以自动适应。